

WEEK-7:

Task 1: Process Synchronization and Child Process Monitoring Using waitpid().

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    int status;
```

```
    printf("Parent process started. PID = %d\n", getpid());
```

```
    pid = fork();
```

```
    if (pid < 0) {
```

```
        printf("Fork failed!\n");
```

```
        return 1;
```

```
    }
```

```
    if (pid == 0) {
```

```
        // Child process
```

```
        printf("Child process started. PID = %d\n", getpid());
```

```
    printf("Child is working...\n");  
    sleep(3);  
  
    printf("Child process completed.\n");  
    exit(0);  
}  
else {  
    // Parent process  
    printf("Parent is waiting for child process...\n");  
  
    waitpid(pid, &status, 0);  
  
    if (WIFEXITED(status)) {  
        printf("Child process terminated normally.\n");  
        printf("Child exit status = %d\n", WEXITSTATUS(status));  
    }  
  
    printf("Parent process completed.\n");  
}  
  
return 0;  
}
```

```
ashwithreddy_earla@AshwithReddy:~$ nano skill7.c
ashwithreddy_earla@AshwithReddy:~$ gcc skill7.c -o skill7
ashwithreddy_earla@AshwithReddy:~$ ./skill7
Parent process started. PID = 563
Parent is waiting for child process...
Child process started. PID = 564
Child is working...
Child process completed.
Child process terminated normally.
Child exit status = 0
Parent process completed.
```

Task 2: To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/stat.h>
```

```
int main() {
```

```
    char *path;
```

```
    char *path_copy;
```

```
    char *directory;
```

```
    char full_path[1024];
```

```
    char command[100];
```

```
    struct stat file_info;
```

```
    int found = 0;
```

```
    // Retrieve PATH variable
```

```
path = getenv("PATH");
```

```
if (path == NULL) {
```

```
    printf("PATH variable not found.\n");
```

```
    return 1;
```

```
}
```

```
printf("PATH Variable:\n%s\n\n", path);
```

```
// Get command from user
```

```
printf("Enter command to search: ");
```

```
scanf("%99s", command);
```

```
// Make a copy because strtok modifies the string
```

```
path_copy = strdup(path);
```

```
if (path_copy == NULL) {
```

```
    printf("Memory allocation failed.\n");
```

```
    return 1;
```

```
}
```

```
// Parse PATH directories
```

```
directory = strtok(path_copy, ":");
```

```
while (directory != NULL) {

    // Create complete path

    snprintf(full_path, sizeof(full_path), "%s/%s",

        directory, command);

    // Check whether file exists

    if (stat(full_path, &file_info) == 0) {

        // Check whether it is a regular file

        if (S_ISREG(file_info.st_mode)) {

            // Verify execute permission

            if (access(full_path, X_OK) == 0) {

                printf("Command found: %s\n", full_path);

                printf("Execute permission: Yes\n");

                found = 1;

                break;

            }

        }

    }

}
```

```

    directory = strtok(NULL, ":");

}

if (!found) {

    printf("Command '%s' not found in PATH.\n", command);

}

free(path_copy);

return 0;

}

```

```

ashwithreddy_earla@AshwithReddy:~$ nano skill7.2.c
ashwithreddy_earla@AshwithReddy:~$ gcc skill7.2.c -o skill7.2
ashwithreddy_earla@AshwithReddy:~$ ./skill7.2
PATH Variable:
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Window
sApps/MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3d8bbwe:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDO
WS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/Program Files/dotnet:/mnt/c
/Program Files/Git/cmd:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin:/mnt/c/Users/ashwithreddy_earla/AppData/Local/Programs/Python/
Python310/Scripts:/mnt/c/Users/ashwithreddy_earla/AppData/Local/Programs/Python/Python310:/mnt/c/Users/ashwithreddy_earla/AppData/L
ocal/Microsoft/WindowsApps:/mnt/c/Users/ashwithreddy_earla/AppData/Local/Programs/Microsoft VS Code/bin:/snap/bin

Enter command to search: ls
Command found: /usr/bin/ls
Execute permission: Yes

```